

Lab 3 : Investigating Process Lifecycles and OS Interaction

Shweta Bhandari

BSc (Hons) Computer Science with AI, Softwarica College of IT and E-Commerce, Coventry
University

ST5003CMD Operating Systems, Security, And Networks

Submitted to : Rupak Rajbanshi

26th September, 2026

Table of contents

I. Introduction.....	3
II. Task 1: The Long-Running Process.....	3
A. C Source Code.....	4
B. Terminal Execution and Verification.....	4
C. Flow Diagram.....	5
III. Task 2 : Process Identity (PID and PPID).....	6
A. Source Code.....	6
B. Terminal Execution.....	6
C. Flow Diagram.....	7
IV. Task 3 : Exit Codes and OS Feedback.....	8
A. Source Code.....	8
B. Execution.....	8
C. Flow Diagram.....	9
V. Task 4: Standard I/O Streams.....	10
A. Source Code.....	10
B. Terminal Output.....	10
C. Flow Diagram.....	11
VI. Task 5: Conditional Execution and Termination.....	12
A. Source Code.....	12
B. Terminal Output.....	12
C. Flow Diagram.....	13
VII. Conclusion.....	14
VIII. GitHub Repository.....	14
Lab3Repo.....	14
IX. Appendix.....	15

List of figures

Figure 1 : Source code.....	5
Figure 2 : Output.....	5
Figure 3: Flow diagram of task1_alive.c.....	6
Figure 4: Source code Identity.c.....	7
Figure 5 : Output.....	7
Figure 6: Flow Diagram of process identity.c.....	8
Figure 7 : Source code exit.c.....	9
Figure 8 : Output.....	10
Figure 9 : Flow diagram of exit.c.....	10
Figure 10: Source code input.c.....	11
Figure 11: Output.....	11
Figure 12: Flow diagram of input.c.....	12
Figure 13 : Source code control.c.....	13
Figure 14: Output.....	14
Figure 15 : Flow chart of control.c.....	14

I. Introduction

In Lecture 3, we learned that an OS manages processes and resources. In this lab, we bring that theory to life. Instead of just writing code that prints text, we will write code that interacts with the Linux kernel to query its own process identity, control its execution time, and report its success or failure back to the OS shell.

All five programs were written in C using the nano editor, compiled with GCC, and executed on Ubuntu 24.04 running under WSL. Each task includes the source code, a flow diagram, the terminal commands used for verification, and screenshots of the output.

II. Task 1: The Long-Running Process

OS Insight: In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

Theory: A process is a program in execution. While a process is running, the kernel keeps it in its process table, where it can be listed with commands such as `ps`. The `sleep()` function suspends the process for a number of seconds; during this time the process is in the sleeping (S) state and uses almost no CPU. Adding `&` to a command runs the process in the background, so the shell stays free for other commands.

A. C Source Code

```
GNU nano 7.2 task1_alive.c
#include <stdio.h>
#include <unistd.h>

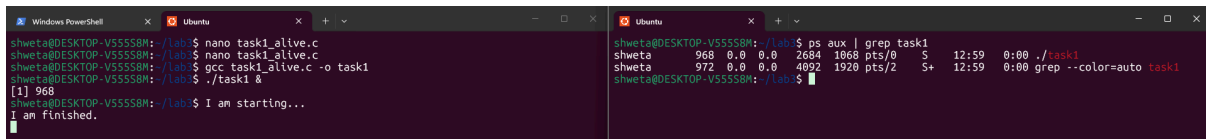
int main() {
    printf("I am starting...\n");

    // Loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }

    printf("I am finished.\n");
    return 0;
}
```

Figure 1 : Source code

B. Terminal Execution and Verification



The figure shows two terminal windows. The left window, titled 'Windows PowerShell' and 'Ubuntu', shows the user 'shweta@DESKTOP-V555S8M' in the directory '/lab3'. The user runs 'nano task1_alive.c', then 'gcc task1_alive.c -o task1', and finally './task1 &'. The output shows '[1] 968' followed by 'shweta@DESKTOP-V555S8M: /lab3\$ I am starting...' and 'I am finished.'. The right window, titled 'Ubuntu', shows the user 'shweta@DESKTOP-V555S8M' in the directory '/lab3' running 'ps aux | grep task1'. The output shows two lines: 'shweta 968 0.0 0.0 2684 1668 pts/0 S 12:59 0:00 ./task1' and 'shweta 972 0.0 0.0 4892 1920 pts/2 S+ 12:59 0:00 grep --color=auto task1'.

Figure 2 : Output

C. Flow Diagram

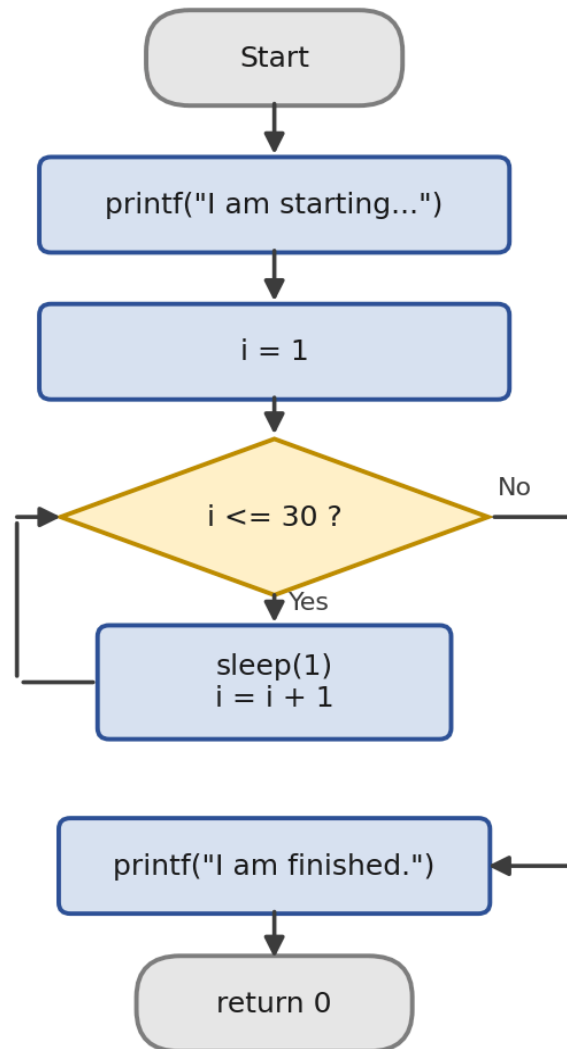


Figure 3: Flow diagram of `task1_alive.c`

Observation: The program was started in the background with `&` and given PID 968. While it was alive, `ps aux | grep task1` listed it with state S (sleeping), confirming that `sleep()` suspends the process instead of using CPU. After 30 seconds it printed "I am finished." and disappeared from the process list.

III. Task 2 : Process Identity (PID and PPID)

OS Insight: Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually your terminal shell.

Theory: The kernel gives every new process a unique PID. A process is always created by another process, so it also stores the PID of its creator as the PPID. The system calls `getpid()` and `getppid()`, declared in `unistd.h`, let a program ask the kernel for these values. Because the program is started from the shell, its PPID is normally the PID of the bash shell.

A. Source Code

```
GNU nano 7.2 task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

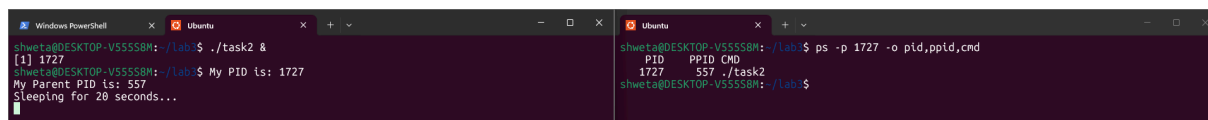
    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 4: Source code Identity.c

B. Terminal Execution



```
shweta@DESKTOP-V55558M: ~/Lab$ ./task2 &
[1] 1727
shweta@DESKTOP-V55558M: ~/Lab$ My PID is: 1727
My Parent PID is: 557
Sleeping for 20 seconds...

shweta@DESKTOP-V55558M: ~/Lab$ ps -p 1727 -o pid,ppid,cmd
PID    PPID CMD
1727   557 ./task2
shweta@DESKTOP-V55558M: ~/Lab$
```

Figure 5 : Output

C. Flow Diagram

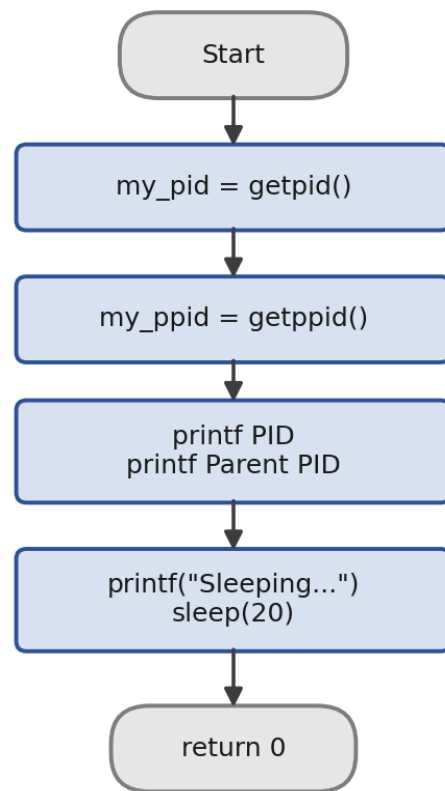


Figure 6: Flow Diagram of process identity.c

Observation: The program reported PID 1727 and Parent PID 557. The command `ps -p 1727 -o pid,ppid,cmd` confirmed the same values, and checking PID 557 showed `-bash`, proving the parent is the shell that started it.

IV. Task 3 : Exit Codes and OS Feedback

OS Insight: When a program finishes, it returns an integer to the OS. 0 means "Success", and any non-zero number (like 1) means "Error/Failure". The shell stores this in \$?.

Theory: The value returned from main() becomes the exit status of the process. The shell keeps the exit status of the last command in the special variable \$?, which is printed with echo \$? . Exit codes allow scripts and automation tools to decide whether the next step should run. The status must be checked immediately after the program, because any other command replaces it.

A. Source Code

```
GNU nano 7.2 task3_exit.c
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

Figure 7 : Source code exit.c

B. Execution

```
shweta@DESKTOP-V555S8M:~/lab3$ nano task3_exit.c
shweta@DESKTOP-V555S8M:~/lab3$ gcc task3_exit.c -o task3
shweta@DESKTOP-V555S8M:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
shweta@DESKTOP-V555S8M:~/lab3$ echo $?
0
shweta@DESKTOP-V555S8M:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
shweta@DESKTOP-V555S8M:~/lab3$ echo $?
1
shweta@DESKTOP-V555S8M:~/lab3$
```

Figure 8 : Output

C. Flow Diagram

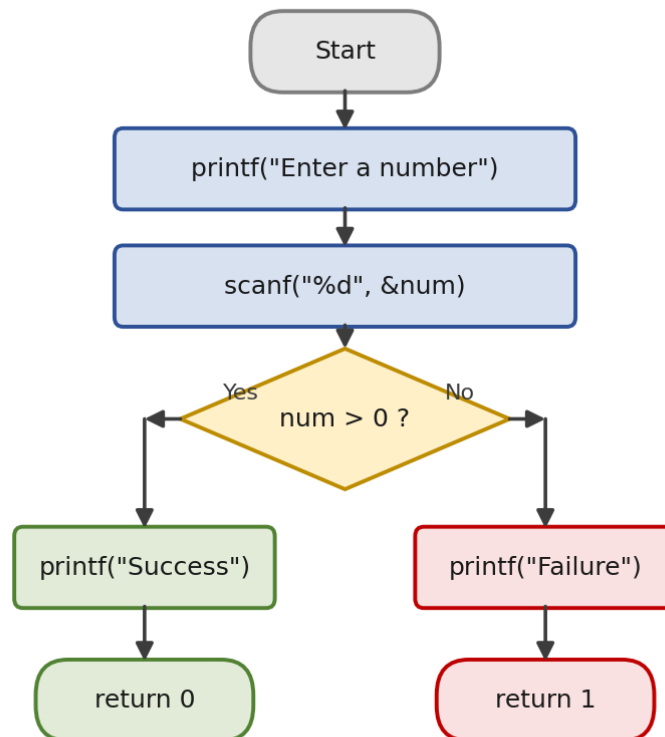


Figure 9 : Flow diagram of exit.c

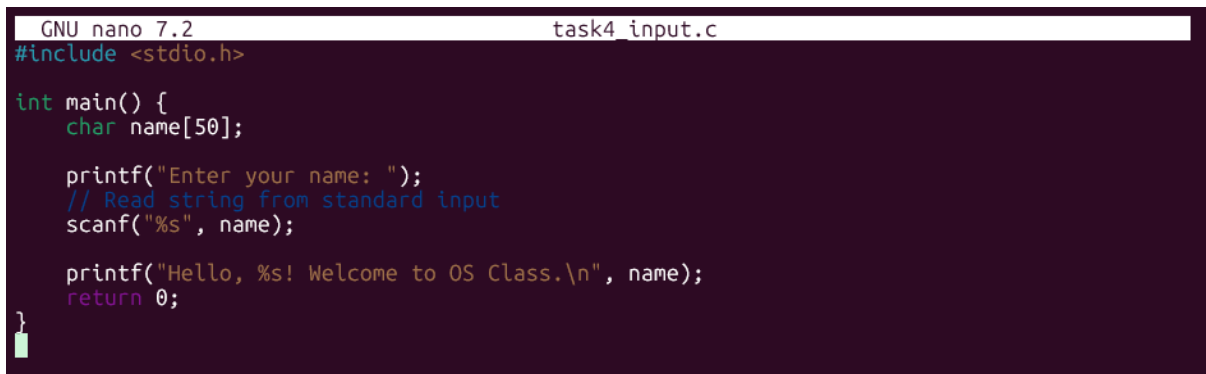
Observation: Entering 5 printed "Success" with exit status 0, and entering -5 printed "Failure" with exit status 1, showing the program reporting its result to the operating system.

V. Task 4: Standard I/O Streams

OS Insight: The OS provides standard input (stdin) and standard output (stdout) streams. scanf reads from stdin, and printf writes to stdout.

Theory: When a process starts, the operating system opens three standard streams for it: standard input (stdin, file descriptor 0), standard output (stdout, 1) and standard error (stderr, 2). By default stdin is connected to the keyboard and stdout to the terminal screen. scanf("%s", name) reads one word from stdin and stores it in the character array, and printf writes to stdout. Because %s stops at the first space, only a single word is stored.

A. Source Code



```
GNU nano 7.2 task4_input.c
#include <stdio.h>

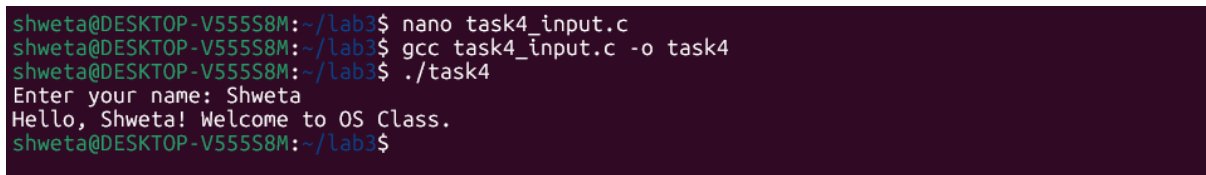
int main() {
    char name[50];

    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```

Figure 10: Source code input.c

B. Terminal Output



```
shweta@DESKTOP-V555S8M:~/lab3$ nano task4_input.c
shweta@DESKTOP-V555S8M:~/lab3$ gcc task4_input.c -o task4
shweta@DESKTOP-V555S8M:~/lab3$ ./task4
Enter your name: Shweta
Hello, Shweta! Welcome to OS Class.
shweta@DESKTOP-V555S8M:~/lab3$
```

Figure 11: Output

C. Flow Diagram

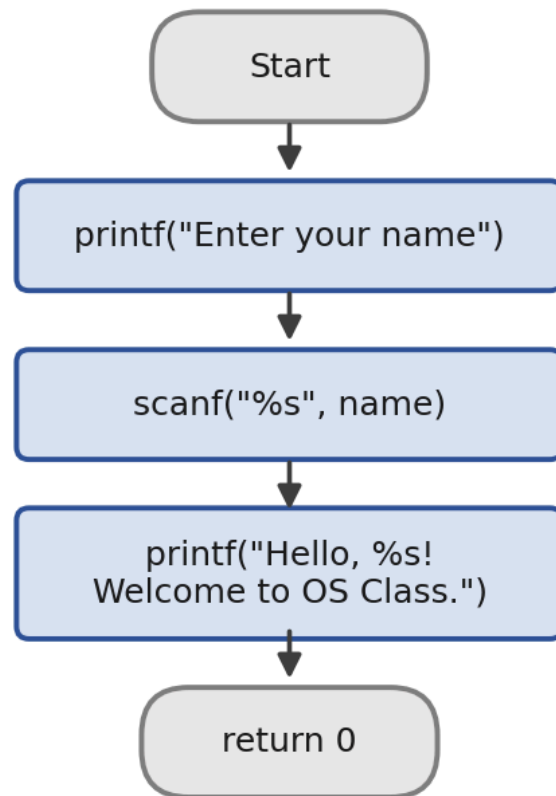


Figure 12: Flow diagram of input.c

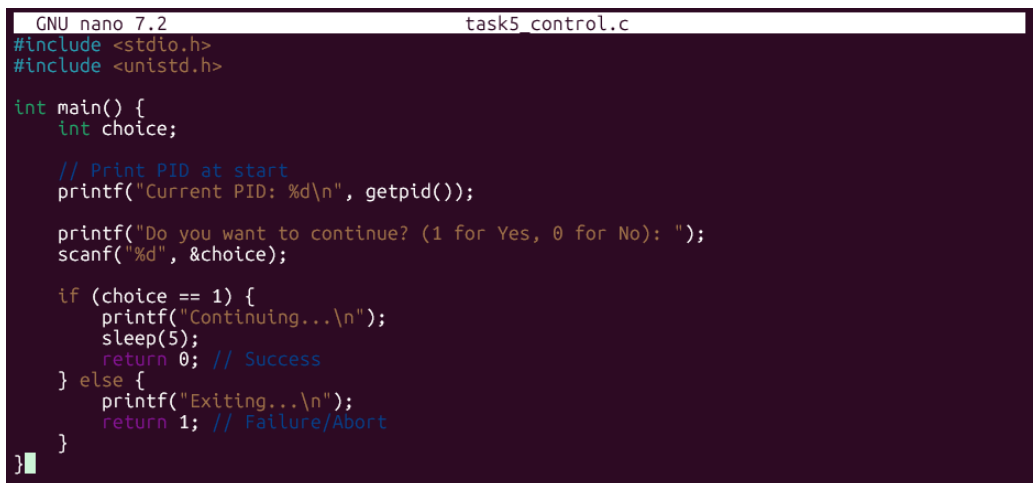
Observation: The program waited at the prompt until a name was typed, showing it was blocked while reading from standard input, then printed the greeting to standard output.

VI. Task 5: Conditional Execution and Termination

OS Insight: Programs often branch based on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.

Theory: This task combines the previous ideas: the program reports its own identity with `getpid()`, reads a choice from standard input, delays with `sleep()`, and ends with an exit code that depends on the branch taken. Each time the program is run, the kernel gives it a new PID, so the two runs show different PID values.

A. Source Code



```
GNU nano 7.2 task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

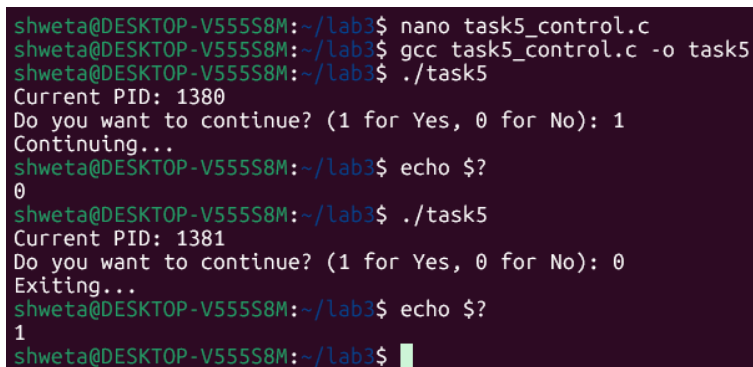
    // Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}
```

Figure 13 : Source code control.c

B. Terminal Output



```
shweta@DESKTOP-V555S8M:~/lab3$ nano task5_control.c
shweta@DESKTOP-V555S8M:~/lab3$ gcc task5_control.c -o task5
shweta@DESKTOP-V555S8M:~/lab3$ ./task5
Current PID: 1380
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
shweta@DESKTOP-V555S8M:~/lab3$ echo $?
0
shweta@DESKTOP-V555S8M:~/lab3$ ./task5
Current PID: 1381
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
shweta@DESKTOP-V555S8M:~/lab3$ echo $?
1
shweta@DESKTOP-V555S8M:~/lab3$
```

Figure 14: Output

C. Flow Diagram

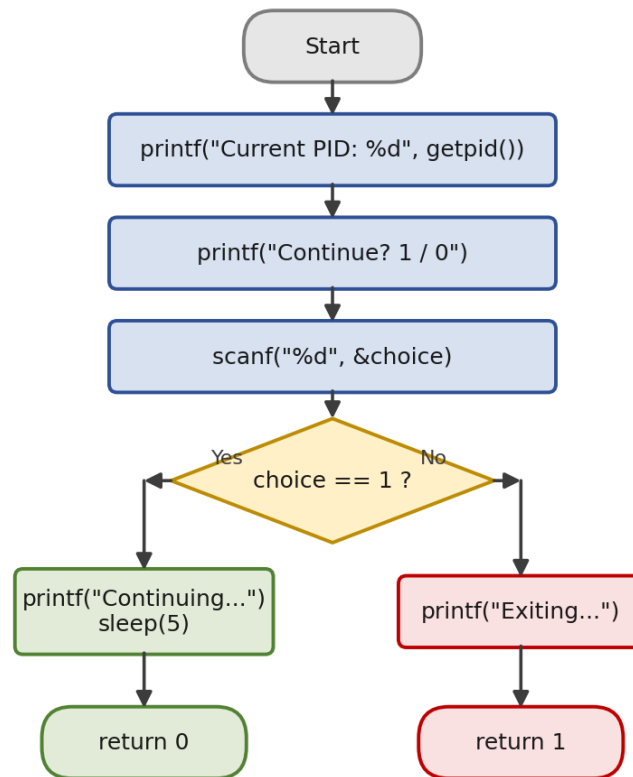


Figure 15 : Flow chart of `control.c`

Observation: Choosing 1 printed "Continuing...", paused for 5 seconds and returned exit status 0, while choosing 0 printed "Exiting..." and returned 1. The PID differed between the two runs, because the kernel assigns a new PID each time a program is executed.

VII. Conclusion

This lab demonstrated that C programs do not exist in a vacuum; they are active processes managed by the Linux kernel. By utilizing system calls like `getpid()`, `sleep()`, and return codes, we successfully bridged the gap between high-level C code and low-level OS process management. Running a program in the background and inspecting it with `ps` showed how the kernel tracks a live process, while the exit status experiments showed how a finished process reports success or failure back to the shell.

VIII. GitHub Repository

[Lab3Repo](#)

IX. Appendix

```
shweta@DESKTOP-V555S8M:~/lab2$ ls
task1      task2      task3      task4      task5
task1_alive.c task2_identity.c task3_exit.c task4_input.c task5_control.c
shweta@DESKTOP-V555S8M:~/lab2$ cd ~/lab2
shweta@DESKTOP-V555S8M:~/lab2$ mkdir lab3_process
shweta@DESKTOP-V555S8M:~/lab2$ cp ~/lab3/*.c lab3_process/
shweta@DESKTOP-V555S8M:~/lab2$ ls lab3_process
task1_alive.c task2_identity.c task3_exit.c task4_input.c task5_control.c
shweta@DESKTOP-V555S8M:~/lab2$ nano lab3_process/README.md
shweta@DESKTOP-V555S8M:~/lab2$ ls lab3_process
README.md task1_alive.c task2_identity.c task3_exit.c task4_input.c task5_control.c
shweta@DESKTOP-V555S8M:~/lab2$ git add .
shweta@DESKTOP-V555S8M:~/lab2$ git commit -m "Add Lab 3: process lifecycles and OS interaction"
[main ccda92c] Add Lab 3: process lifecycles and OS interaction
6 files changed, 147 insertions(+)
create mode 100644 lab3_process/README.md
create mode 100644 lab3_process/task1_alive.c
create mode 100644 lab3_process/task2_identity.c
create mode 100644 lab3_process/task3_exit.c
create mode 100644 lab3_process/task4_input.c
create mode 100644 lab3_process/task5_control.c
shweta@DESKTOP-V555S8M:~/lab2$ git push
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 2.54 KiB | 1.27 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/swt-b/ST5003CMD-Lab2.git
4416296..ccda92c main -> main
shweta@DESKTOP-V555S8M:~/lab2$
```

```
Windows PowerShell  X  Ubuntu  X  +  v
shweta@DESKTOP-V555S8M:~/lab2$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

shweta@DESKTOP-V555S8M:~/lab2$
```